

Core intervention concepts

Targets, effects, atoms, programs, and epochs

This vignette introduces the basic data model in `CategoricalInterventions.jl`. An intervention is a local edit to a named target over a half-open interval `[start, stop)`.

```
using CategoricalInterventions

beta = Target(:beta; kind=Parameter, value_type=Float64)
gamma = Target(:gamma; kind=Parameter, value_type=Float64)

school_closure = InterventionAtom(
    :school_closure,
    beta,
    Interval(10, 30),
    Scale(0.6),
)

case_isolation = InterventionAtom(
    :case_isolation,
    gamma,
    Interval(20, 40),
    SetValue(0.25),
)

program = compose_interventions(
    InterventionProgram(school_closure),
    InterventionProgram(case_isolation),
)
```

`InterventionProgram(2 atoms)`

The same objects can be written with Unicode constructors and operators. In the Julia REPL, type the LaTeX-style name and press TAB, such as `\bbP<TAB>` for `Program` or `\oplus<TAB>` for `plus`.

```

= (:beta; value_type=Float64, algebra=MultiplicativeAlgebra())
= (:gamma; value_type=Float64)

closure = (:school_closure, , (10, 30), (0.6))
isolation = (:case_isolation, , (20, 40), (0.25))

unicode_program =  $\Pi$ (closure) isolation
unicode_program Dict(:beta => 0.30, :gamma => 0.10)

```

```

3-element Vector{EpochValues}:
 EpochValues([10, 20], Dict{Symbol, Any}(:beta => 0.18, :gamma => 0.1), InterventionAtom[sch
 EpochValues([20, 30], Dict{Symbol, Any}(:beta => 0.18, :gamma => 0.25), InterventionAtom[sch
 EpochValues([30, 40], Dict{Symbol, Any}(:beta => 0.3, :gamma => 0.25), InterventionAtom[case

```

Overlapping interventions on different targets are independent. The common refinement of their support intervals creates epochs.

```

epochs = epochize(program)
[(epoch.support, collect(keys(epoch.effects))) for epoch in epochs]

```

```

3-element Vector{Tuple{Interval{Int64}, Vector{Symbol}}}:
 ([10, 20], [:beta])
 ([20, 30], [:beta, :gamma])
 ([30, 40], [:gamma])

```

Applying a program to a baseline parameter dictionary evaluates the effects in each active epoch.

```

baseline = Dict(:beta => 0.30, :gamma => 0.10)
applied = apply_interventions(program, baseline)

[(epoch.support, epoch.values) for epoch in applied]

```

```

3-element Vector{Tuple{Interval{Int64}, Dict{Symbol, Any}}}:
 ([10, 20], Dict(:beta => 0.18, :gamma => 0.1))
 ([20, 30], Dict(:beta => 0.18, :gamma => 0.25))
 ([30, 40], Dict(:beta => 0.3, :gamma => 0.25))

```

The key convention is that intervals are half-open: the effect is active at the start time and inactive at the stop time.

```
CategoricalInterventions.contains_time(Interval(10, 30), 10),  
CategoricalInterventions.contains_time(Interval(10, 30), 30)
```

(true, false)